THE OPEN UNIVERSITY OF SRI LANKA

DEPARTMENT OF ELECTRICAL & COMPUTER ENGINEERING

BACHELOR OF SOFTWARE ENGINEERING

EEI6373 – Performance Modelling

Mini Project

Performance Modeling of a Webhook Delivery System Under Increasing Workload and Worker Scalability

Name: W.K.V.P Randunu

S_Number: S92069544

Reg_No: 721429544

Due Date: 2026/08/10

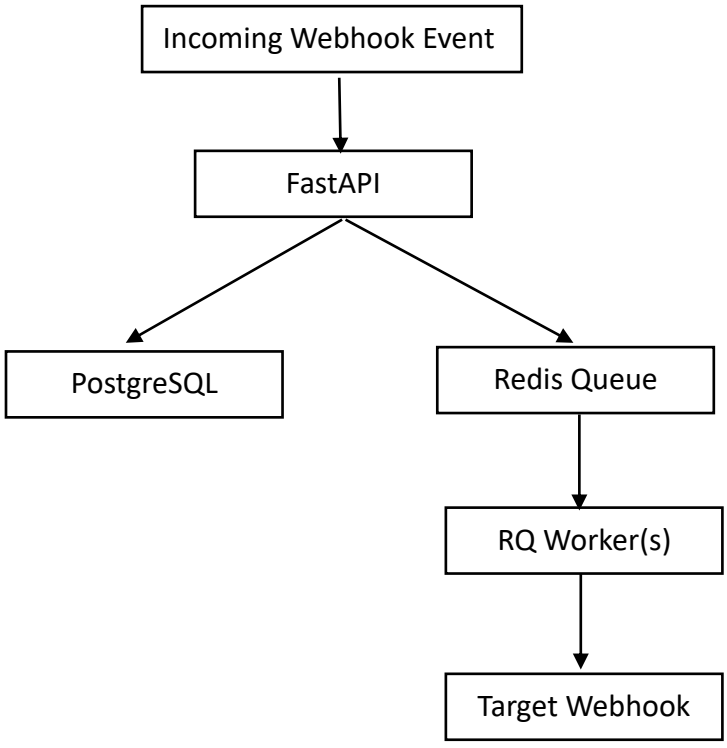
1. System Description and Problem Statement

1.1 System Description

The system considered in this study is a webhook delivery system developed in an earlier software engineering project. The purpose of the system is to receive events from clients and reliably deliver those events to external webhook endpoints.

The system follows an asynchronous event processing architecture. When an event is received through the FastAPI application, the event is persisted in PostgreSQL and a delivery job is placed in a Redis based queue. RQ workers retrieve jobs from the queue and perform HTTP requests to the target webhook endpoint. This separates event reception from webhook delivery and allows delivery work to be processed asynchronously.

At a high level, the system can be represented as:



The webhook delivery system uses a worker based processing model in which jobs waiting in the Redis queue are processed by RQ workers. This architecture can therefore be viewed as a queueing system: incoming events represent the workload entering the system, Redis represents the waiting queue, and the workers represent the available service resources.

The performance study focuses on this core processing path, rather than on the later Kubernetes deployment work. The Kubernetes deployment is therefore outside the scope of the current performance model.

1.2 Problem Statement

As the number of incoming webhook events increases, the available processing capacity of the worker may become insufficient to process events at the same rate at which they arrive. When this occurs, events spend more time waiting in the queue before being processed, resulting in increased delivery latency and potentially reduced system responsiveness.

The implemented system uses a single worker, providing a useful baseline for investigating this problem. However, a performance model can be used to evaluate alternative worker configurations without requiring the physical deployment of every possible configuration.

Therefore, this study investigates how the performance of the webhook delivery system changes under different workload levels and different numbers of processing workers. The main focus is to understand the relationship between incoming workload, processing capacity, queueing behavior, latency, throughput, and worker utilization.

The central performance question of this study is:

How does increasing workload affect the performance of the webhook delivery system, and how does increasing the number of workers affect its ability to handle that workload?

2. Performance Objectives

The main objective of this study is to evaluate how the performance of the webhook delivery system changes as the incoming workload increases and as the number of workers is changed. The study focuses on the following performance measures.

1. Latency
2. Throughput
3. Queue Length
4. Worker Utilization and Scalability
5. Overall Performance Objective

2.1 Latency

The study measures the time required for a webhook event to be processed. It also examines how this time changes as the number of incoming events increases and the workers become more heavily loaded.

2.2 Throughput

The study measures the rate at which webhook events are successfully processed. This helps determine how much workload the system can handle with different numbers of workers.

2.3 Queue Length

The study examines the number of events waiting to be processed in the Redis queue. Increasing queue length can indicate that incoming workload is exceeding the available processing capacity.

2.4 Worker Utilization and Scalability

The study evaluates how worker utilization changes under different workload levels and how increasing the number of workers affects system performance. This helps determine whether adding more workers provides meaningful performance improvements.

2.5 Overall Performance Objective

Overall, the study aims to identify the relationship between workload, worker capacity, queueing behavior, latency, and throughput, and to determine how alternative worker configurations can improve the system's ability to handle increasing workload.

Latency is used as the primary metric for visual comparison because it directly shows the effect of workload and worker capacity on the time required to process an event. Throughput, queue length, and worker utilization are used as supporting measures to explain the changes observed in latency and to identify conditions where the system approaches its processing capacity.

3. Modeling Approach and Assumptions

3.1 Queueing Model

The webhook delivery system uses Redis as a queue for delivery jobs. RQ workers retrieve these jobs and process them by sending requests to the target webhook endpoint. This structure can be represented as a multi server queue, where incoming webhook events form the workload, Redis provides the waiting queue, and the workers provide the available processing capacity.

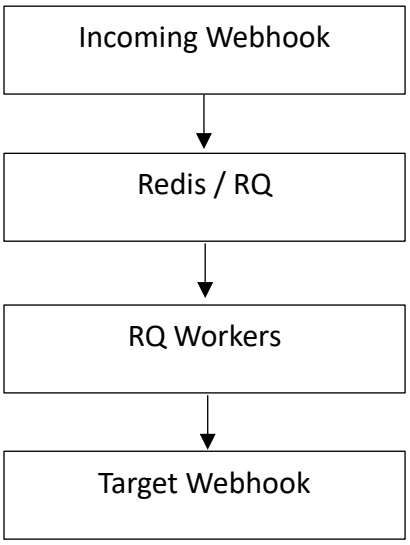


Figure 1. Simplified performance model of the webhook delivery system.

Based on this structure, an **M/M/c queueing model** is used to represent the worker processing part of the system. In this model, the first **M** represents a Poisson arrival process, the second **M** represents exponentially distributed service times, and **c** represents the number of available workers.

The main parameters used in the model are:

- **λ (lambda)** — the rate at which webhook events arrive.
- **μ (mu)** — the service rate of a single worker.
- **c** — the number of workers available to process events.
- **ρ (rho)** — the utilization of the available worker capacity.

The utilization is calculated as:

$$\rho = \lambda / c\mu$$

This model allows the relationship between workload and processing capacity to be studied as the arrival rate and number of workers are changed.

3.2 Discrete Event Simulation

In addition to the analytical queueing model, a discrete event simulation will be used to evaluate the system under different workload and worker configurations.

The simulation represents webhook events arriving over time, waiting when all workers are busy, and being processed when a worker becomes available. Different arrival rates and worker counts can then be tested without having to physically deploy every configuration.

The simulation was used to generate performance results such as latency, throughput, queue length, and worker utilization. These results can then be compared across different workload levels and worker configurations.

3.3 Assumptions

The performance model uses the following assumptions:

1. **Independent arrivals:** Webhook events are assumed to arrive independently according to the selected arrival rate.
2. **Similar workers:** Each worker is assumed to have approximately the same processing capacity.
3. **Queue availability:** The Redis queue is assumed to have sufficient capacity for the workload considered in the simulation.

4. **Constant service characteristics:** Each worker is assigned a defined service rate for the simulation.
5. **Focus on worker processing:** The model focuses primarily on the queue and worker-processing stage rather than modelling every component of the complete infrastructure.
6. **Controlled workload:** Arrival rates and worker counts are varied deliberately so that their effects on system performance can be observed.

4. Dataset and Methodology

4.1 Dataset

The performance dataset used in this study was generated using a simulation of the webhook delivery system. The implemented system was not tested with a planned range of workloads and worker configurations, so the available monitoring data does not provide enough controlled data for the experiments in this study. The simulation therefore allows different workload and worker configurations to be tested and compared.

The dataset contains results for different arrival rates and numbers of workers. For each combination, the simulation will record the following performance measures:

- Arrival rate
- Number of workers
- Processing latency
- Throughput
- Queue length
- Worker utilization

The simulated dataset allows the performance of the system to be compared under increasing workloads and different levels of processing capacity.

4.2 Methodology

The study used controlled experiments to investigate the effect of workload and worker capacity on system performance.

First, the arrival rate of webhook events will be varied while keeping the number of workers fixed. This will be used to measure how increasing workload affects latency, throughput, queue length, and worker utilization.

Experiment 1:

Change **workload** → keep workers fixed → measure performance.

Second, the number of workers will be varied while keeping the workload fixed. This will be used to evaluate how increasing processing capacity affects system performance.

Experiment 2:

Change **workers** → keep workload fixed → measure performance.

The results from each experiment will be recorded and compared using tables and graphs. The results will then be used to identify performance trends and determine when increasing workload causes higher latency, longer queues, or reduced throughput.

4.3 Experimental Variables

The main variables used in the experiments are:

Variable	Type	Description
Arrival rate (λ)	Independent	Number of webhook events arriving per second
Number of workers (c)	Independent	Number of workers available to process events
Latency	Dependent	Time required to process an event
Throughput	Dependent	Number of events successfully processed per second
Queue length	Dependent	Number of events waiting for processing
Worker utilization	Dependent	Percentage of available worker capacity being used

5. Experiments

The experiments are designed to study how workload and worker capacity affect the performance of the webhook delivery system. Three experiments were conducted.

5.1 Experiment 1 — Effect of Increasing Workload

The first experiment examines how the system performs as the workload increases.

The number of workers will be kept at one while the arrival rate of webhook events is gradually increased. For each arrival rate, the simulation will record latency, throughput, queue length, and worker utilization.

The results will be used to understand how the system behaves as the workload increases and how close the single worker configuration gets to its processing capacity.

5.2 Experiment 2 — Effect of Increasing the Number of Workers

The second experiment examines how adding workers affects system performance.

The arrival rate will be kept at a fixed level while the number of workers is increased. For each worker configuration, the simulation will record latency, throughput, queue length, and worker utilization.

The results will be compared to determine whether adding more workers improves performance and whether the improvement becomes smaller as more workers are added.

5.3 Experiment 3 — Increasing Workload and Worker Capacity Together

The third experiment examines whether the system can maintain its performance when both workload and processing capacity are increased.

In this experiment, the arrival rate and the number of workers will be increased together. For each combination, the simulation will record latency, throughput, queue length, and worker utilization.

The results will be used to determine whether increasing the number of workers can keep up with the increase in workload and maintain stable system performance.

5.4 Comparison of Results

The results from all three experiments will be compared using tables and graphs. The comparison will focus on latency, throughput, queue length, and worker utilization.

The results will be used to identify how the system behaves under increasing workload, how much benefit is gained by adding workers, and whether worker capacity can be increased along with workload to maintain acceptable performance.

6. Results and Visualizations

The simulation recorded latency, throughput, mean queue length, and worker utilization for each experimental configuration. Latency is used as the main visual measure in the following experiments, while the other metrics are used to support the interpretation of system behavior.

6.1 Experiment 1 — Effect of Increasing Workload

The first experiment examined how the performance of the system changed as the arrival rate increased while keeping the number of workers at one.

The simulation showed that latency increased gradually at lower workload levels but increased significantly as the arrival rate approached and exceeded the service capacity of the single worker. At an arrival rate of 8 events per second, the mean latency was approximately **0.52 seconds**, increasing to approximately **0.98 seconds** at 9 events per second. When the arrival rate reached

10 events per second, the mean latency increased to approximately **8.21 seconds**. At 11 and 12 events per second, the mean latency increased further to approximately **52.54 seconds** and **99.41 seconds**, respectively.

This shows that the system becomes increasingly sensitive to workload as the arrival rate approaches the processing capacity of the worker.

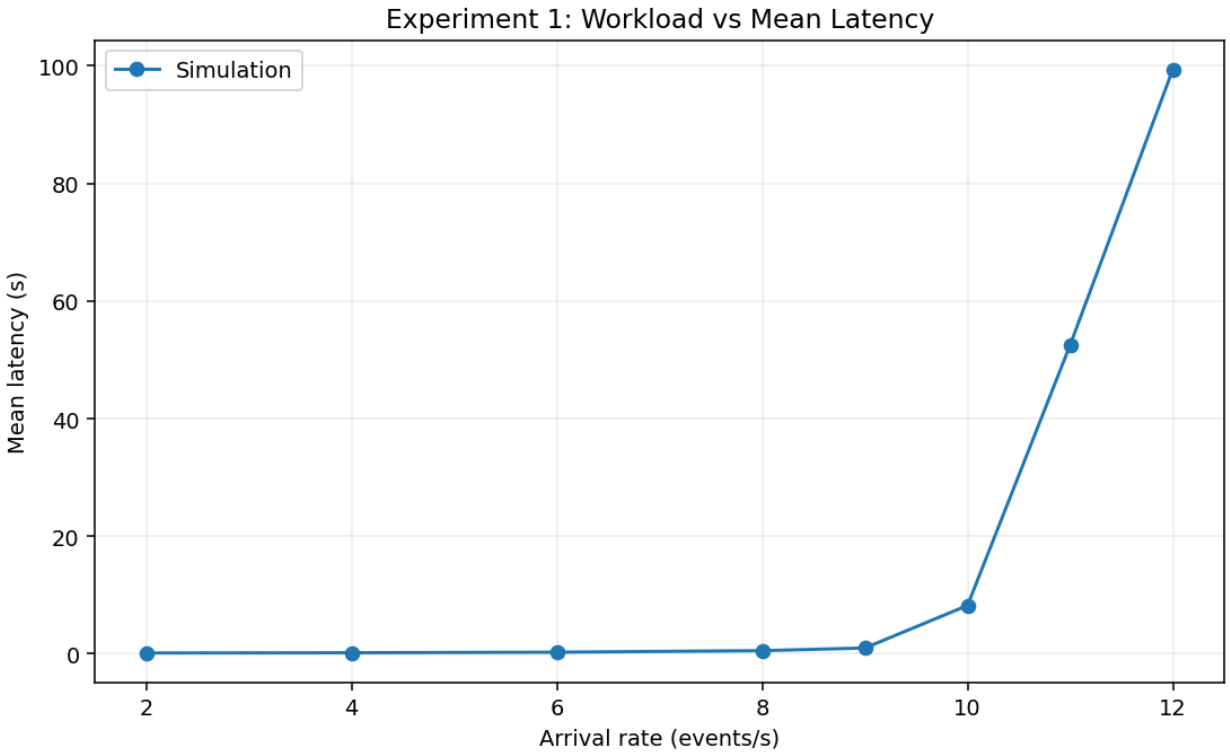


Figure 2. Effect of increasing workload on mean latency with one worker.

6.2 Experiment 2 — Effect of Increasing the Number of Workers

The second experiment examined the effect of increasing the number of workers while keeping the arrival rate fixed at 8 events per second.

With one worker, the mean latency was approximately **0.52 seconds**. Increasing the number of workers to two reduced the mean latency to approximately **0.12 seconds**. Further increasing the worker count to three and four resulted in mean latencies of approximately **0.10 seconds** in both cases.

The results show that adding a second worker provides a substantial improvement under the selected workload. However, the additional improvement from the third and fourth workers is much smaller.

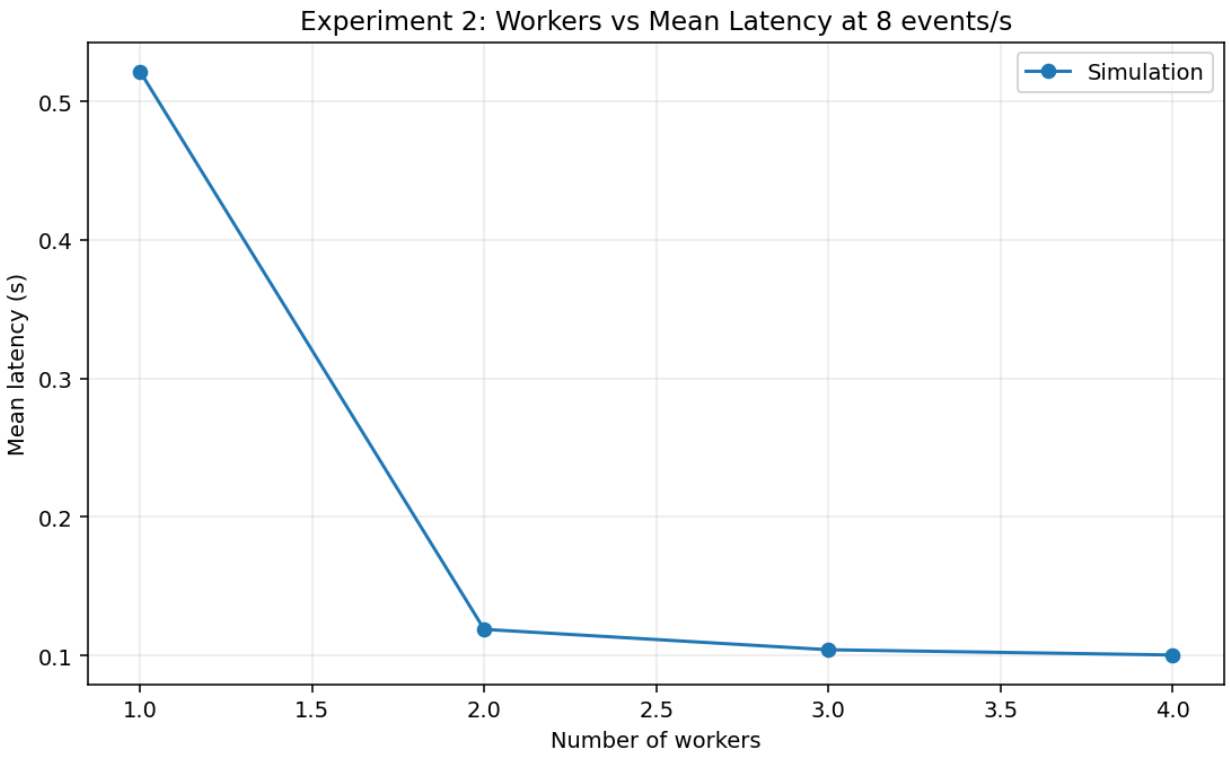


Figure 3. Effect of increasing the number of workers on mean latency at an arrival rate of 8 events/s.

6.3 Experiment 3 — Increasing Workload and Worker Capacity Together

The third experiment examined whether the system could maintain its performance when workload and worker capacity were increased together.

The workload was increased from 8 to 32 events per second while the number of workers was increased from one to four. The mean latency decreased from approximately **0.52 seconds at 8 events/s with one worker** to approximately **0.17 seconds at 32 events/s with four workers**.

The results indicate that increasing worker capacity alongside workload prevented the sharp increase in latency observed in the single-worker experiment.

Figure 4. Mean latency when workload and worker capacity are increased together.

6.4 Summary of Results

The results from the three experiments show a clear relationship between workload, worker capacity, and system performance. In the first experiment, increasing the workload with a single worker caused latency to increase rapidly as the arrival rate approached and exceeded the worker's assumed service capacity. In the second experiment, adding a second worker significantly reduced latency at the selected workload, while adding further workers produced smaller improvements. In the third experiment, increasing the number of workers together with the workload prevented the sharp increase in latency observed in the single worker configuration.

These results indicate that worker capacity has a significant effect on the ability of the system to handle increasing workload. The simulation also shows that simply adding more workers does not provide the same level of improvement at every workload, making the relationship between workload and available processing capacity an important factor in system performance.

Table 1. Summary of selected simulation results

Experiment	Arrival Rate (events/s)	Workers	Mean Latency (s)
Increasing workload	8	1	0.52
Increasing workload	10	1	8.21
Increasing workload	12	1	99.41
Increasing workers	8	2	0.12
Increasing workers	8	4	0.10
Scaling with workload	16	2	0.28
Scaling with workload	32	4	0.17

7. Analysis and Findings

7.1 Effect of Workload on Performance

The first experiment shows that increasing the arrival rate has a significant effect on system latency when only one worker is available. At lower arrival rates, the worker has enough processing capacity to handle the incoming workload, so latency remains relatively low. As the arrival rate approaches the assumed service rate of 10 events per second, the worker becomes increasingly utilized and more events begin to wait for processing. This results in a rapid increase in latency.

When the arrival rate exceeds the service capacity of the worker, the incoming workload is greater than the rate at which events can be processed. This causes the queue to grow and results in a substantial increase in latency. The results therefore show that workload approaching the available processing capacity is an important performance bottleneck.

7.2 Effect of Increasing Workers

The second experiment shows that increasing the number of workers can significantly reduce latency when the system is under load. At an arrival rate of 8 events per second, increasing the worker count from one to two reduced the mean latency from approximately 0.52 seconds to 0.12 seconds. Further increases to three and four workers resulted in smaller improvements, with mean latency remaining close to 0.10 seconds.

This indicates that adding processing capacity is most beneficial when the existing worker capacity is under significant load. Once sufficient capacity is available for the workload, adding more workers provides smaller improvements.

7.3 Effect of Scaling Workers with Workload

The third experiment shows how the system behaves when worker capacity is increased together with workload. The arrival rate was increased from 8 to 32 events per second while the number of workers increased from one to four. Unlike the single worker experiment, the system did not show the sharp increase in latency that occurred when workload exceeded the capacity of one worker.

This suggests that increasing processing capacity alongside workload can help the system handle increasing demand without the same level of queueing and latency growth. The results demonstrate the importance of matching available worker capacity with the workload placed on the system.

7.4 Supporting Performance Measures

In addition to latency, the simulation recorded throughput, mean queue length, and worker utilization. These measures provide additional information about the behavior of the system as workload changes.

Throughput increased as the arrival rate increased while the worker had sufficient processing capacity. However, once the workload approached the worker's capacity, throughput could no longer increase at the same rate as the incoming workload. This indicates that the worker had reached a practical processing limit.

Queue length is closely related to the increase in latency observed in the first experiment. As the worker becomes more heavily loaded, incoming events are more likely to wait before processing. This increases the number of events waiting in the queue and contributes to higher end-to-end latency.

Worker utilization also increased as the arrival rate increased. High utilization indicates that the available processing capacity is being used heavily. While high utilization can represent efficient resource use, operating too close to full capacity leaves less room to absorb additional workload and can result in rapidly increasing queueing delay.

These supporting measures therefore help explain the latency results and provide a broader view of system performance.

7.5 Overall Findings

Overall, the experiments show that system performance depends strongly on the relationship between incoming workload and available processing capacity. Increasing workload without increasing capacity can cause queueing and rapidly increasing latency once the system approaches saturation. Increasing the number of workers provides additional processing capacity

and can significantly improve performance when the existing capacity is insufficient. However, the benefit of adding workers becomes smaller when sufficient capacity is already available.

The results therefore support the use of worker scaling as a way of handling increased workload in the webhook delivery system. The simulation also demonstrates how a queueing model can be used to evaluate different capacity configurations before deploying them in the actual system.

8. Limitations and Future Work

This study has several limitations. First, the performance results are based on a simulation rather than measurements collected from the deployed webhook delivery system. The service rate used in the model was therefore treated as an assumption rather than a value measured from the actual system.

Second, the simulation focuses on the queue and worker processing part of the webhook delivery system. Other factors such as network latency, database performance, Redis performance, and the performance of the target webhook endpoint are not modelled in detail.

Third, the experiments use a limited number of workload and worker configurations. The results therefore describe the behavior of the system under the conditions tested and should not be treated as a direct prediction of the performance of the production system.

Future work could extend the model by collecting controlled measurements from the deployed system and using those measurements to calibrate the simulation. The model could also be extended to investigate additional factors such as webhook failure rates, retry behavior, network latency, and automatic worker scaling.

9. Conclusion

This study evaluated the performance of a webhook delivery system using an M/M/c queueing model and a discrete event simulation. The system was modelled as a queue in which incoming webhook events are processed by a set of workers. The study focused on the effects of increasing workload and changing the number of available workers on latency, throughput, queueing behavior, and worker utilization.

The simulation results showed that increasing workload while keeping a single worker caused latency to increase significantly as the arrival rate approached and exceeded the assumed service capacity. Increasing the number of workers reduced latency under higher workloads, although the improvement became smaller after sufficient processing capacity was available. The third experiment also showed that increasing worker capacity together with workload can help the system handle increasing demand without the sharp latency growth seen in the single-worker configuration.

Overall, the study demonstrates how queueing models and simulation can be used to evaluate performance and scalability before deploying different configurations of a system. For the webhook delivery system, the results highlight the importance of matching worker capacity with workload and provide a basis for further investigation using measurements from the deployed system.

10. References

[1] The Open University of Sri Lanka, *EEI6373 – Performance Modelling: Mini Project 2025/2026*, Department of Electrical & Computer Engineering, 2026.

[2] SimPy, *SimPy Documentation: Discrete-event simulation for Python*. [Online]. Available: https://simpy.readthedocs.io/

[3] Python Software Foundation, *Python 3 Documentation*. [Online]. Available: https://docs.python.org/3/

[4] RQ, *RQ Documentation: Simple job queues for Python*. [Online]. Available: https://python-rq.org/

[5] Redis Ltd., *Redis Documentation*. [Online]. Available: https://redis.io/docs/

[6] W. J. Stewart, *Probability, Markov Chains, Queues, and Simulation: The Mathematical Basis of Performance Modeling*. Princeton University Press, 2009.